

UDP Packet Sender

Report della mia esercitazione: invio di pacchetti UDP con Python

1. Obiettivo dell'Esercizio

Creare uno script Python che invii un numero configurabile di pacchetti UDP da 1 KB verso un host e una porta specificati dall'utente. L'esercizio introduce i socket di rete, il protocollo UDP e la generazione di dati binari casuali.

2. Concetti Fondamentali

2.1 - Il protocollo UDP

Tipo	Connectionless: nessuna connessione preventiva
Garanzie	Nessuna: non c'e' ACK, non c'e' riordinamento, nessun retry
Velocita'	Molto alta rispetto a TCP grazie all'assenza di overhead
Uso comune	Streaming video/audio, DNS, VoIP, giochi online, testing di rete
Uso in sicurezza	UDP Flood (DoS), fuzzing di servizi, port scanning stealth

2.2 - Socket di rete in Python

AF_INET	Address Family Internet: usa indirizzi IPv4 (es. 192.168.1.1)
SOCK_DGRAM	Socket di tipo datagramma: corrisponde al protocollo UDP
SOCK_STREAM	Socket di tipo stream: corrisponde al protocollo TCP (NON usato qui)
sendto()	Invia bytes a (ip, porta) senza connessione preventiva
close()	Rilascia il descrittore di file del socket dal sistema operativo

2.3 - Generazione di dati casuali

randbytes(n)	Genera n byte casuali con valori 0-255 (Python 3.9+)

1 KB	1 Kilobyte = 1024 byte; dimensione standard dell'esercizio
Perche' casuale?	Simula payload reale e non viene compresso o filtrato facilmente

3. Struttura del Programma

Ho strutturato il programma seguendo un flusso lineare in 6 fasi:

Fase 1	Input IP target	input() legge la stringa inserita dall'utente. L'IP non viene validato nell'esercizio base.
Fase 2	Input porta target	int(input()) converte l'input in intero. Le porte valide sono 0-65535.
Fase 3	Input numero pacchetti	Determina quante iterazioni eseguirà il ciclo di invio.
Fase 4	Creazione socket UDP	socket.socket(AF_INET, SOCK_DGRAM) apre il canale di comunicazione.
Fase 5	Ciclo di invio	Per ogni iterazione: genera 1024 byte casuali e li invia con sendto().
Fase 6	Chiusura socket	sock.close() libera le risorse di sistema. Buona pratica sempre.

4. Codice Sorgente Commentato

```
import socket
import random

# 1. Input target
ip_target = input("Inserisci l'IP della macchina target: ")
porta_target = int(input("Inserisci la porta UDP: "))
num_pacchetti = int(input("Quanti pacchetti da 1 KB vuoi inviare? "))

# 2. Creazione socket UDP
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# 3. Ciclo di invio
for i in range(num_pacchetti):
    pacchetto = random.randbytes(1024) # 1 KB di byte casuali
    sock.sendto(pacchetto, (ip_target, porta_target)) # invio UDP
    print(f"[+] Pacchetto {i+1}/{num_pacchetti} inviato ({len(pacchetto)} byte)")

# 4. Chiusura socket
sock.close()
print("[OK] Invio completato.")
```

5. Funzioni Python Chiave

<code>socket.socket(AF, TYPE)</code>	socket	Crea un socket. AF=famiglia indirizzi, TYPE=tipo di socket.
<code>sock.sendto(data, addr)</code>	socket	Invia bytes all'indirizzo (ip, porta) via UDP. Non richiede connessione.
<code>sock.close()</code>	socket	Chiude il socket e rilascia il file descriptor.
<code>random.randbytes(n)</code>	random	Restituisce n byte casuali. Disponibile da Python 3.9.
<code>input(prompt)</code>	builtins	Stampa prompt e legge una riga da stdin come stringa.
<code>int(x)</code>	builtins	Converte x in intero. Utile per porte e contatori.
<code>range(n)</code>	builtins	Genera la sequenza 0, 1, ..., n-1. Usata nel ciclo for.

6. Note Etiche e di Sicurezza

Questo script e' destinato esclusivamente a scopo didattico e a test su macchine di cui si ha esplicita autorizzazione (es. VM locali, lab Epicode, DVWA). Inviare pacchetti UDP verso host non autorizzati e' illegale in Italia ai sensi dell'art. 615-ter e 617-quater del Codice Penale.

Per testare lo script in sicurezza: usare Kali Linux in VM come source e un'altra VM (es. Metasploitable) come target, su rete host-only. Monitorare il traffico con Wireshark per osservare i pacchetti UDP in transito.

7. Verifica con Wireshark

Per verificare che i pacchetti UDP arrivassero effettivamente alla macchina target e' stato usato Wireshark sul Windows host con la VM Kali in modalita' Host-only. La configurazione usata:

Windows (target)	192.168.56.1 - Host fisico, scheda Ethernet
Porta	443 UDP
Filtro Wireshark	ip.src == 192.168.56.102
Pacchetti inviati	1000 da 2048 byte ciascuno
Frammentazione	Ogni pacchetto da 2048 byte viene frammentato in 2 segmenti da ~1514 byte perche' supera il MTU (1500 byte)

Screenshot della mia cattura Wireshark:

Cattura da Ethernet						
File Modifica Visualizza Vai Cattura Analizza Statistiche Telefonia Wireless Strumenti Aiuto						
ip.src == 192.168.56.102						
No.	Time	Source	Destination	Protocol	Length	Info
2170	390.631253800	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7539) [Reassembled in #2171]
2171	390.631364000	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2172	390.631571400	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=753a) [Reassembled in #2173]
2173	390.631671900	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2174	390.631823800	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=753b) [Reassembled in #2175]
2175	390.631911400	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2176	390.632013600	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=753c) [Reassembled in #2177]
2177	390.632100700	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2178	390.632222500	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=753d) [Reassembled in #2179]
2179	390.632314300	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2180	390.632414000	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=753e) [Reassembled in #2181]
2181	390.632500700	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2182	390.632598900	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=753f) [Reassembled in #2183]
2183	390.632690500	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2184	390.632791300	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7540) [Reassembled in #2185]
2185	390.632877800	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2186	390.632996700	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7541) [Reassembled in #2187]
2187	390.633083600	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2188	390.633202600	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7542) [Reassembled in #2189]
2189	390.633307200	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2190	390.633434500	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7543) [Reassembled in #2191]
2191	390.633521100	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2192	390.633640600	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7544) [Reassembled in #2193]
2193	390.633728200	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2194	390.633849500	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7545) [Reassembled in #2195]
2195	390.634035700	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2196	390.634224300	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7546) [Reassembled in #2197]
2197	390.634316800	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2198	390.634421900	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7547) [Reassembled in #2199]
2199	390.634511600	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2200	390.634615100	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7548) [Reassembled in #2201]
2201	390.634711100	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2202	390.634815700	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7549) [Reassembled in #2203]
2203	390.634906300	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2204	390.635030700	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=754a) [Reassembled in #2205]
2205	390.635122600	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2206	390.635311100	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=754b) [Reassembled in #2207]
2207	390.635403200	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2208	390.635533100	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=754c) [Reassembled in #2209]
2209	390.635622200	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2210	390.635727000	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=754d) [Reassembled in #2211]
2211	390.635818700	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2212	390.635945200	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=754e) [Reassembled in #2213]
2213	390.636038400	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2214	390.636142400	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=754f) [Reassembled in #2215]
2215	390.636232300	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2216	390.636478200	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7550) [Reassembled in #2217]
2217	390.636577900	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048
2218	390.636710700	192.168.56.102	192.168.56.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=7551) [Reassembled in #2219]
2219	390.636801900	192.168.56.102	192.168.56.1	UDP	610	38559 → 443 Len=2048

Screenshot del mio codice e output terminale:


```

venv > udp.py > ...
1  import socket
2  import random
3
4  ip_target = input("Inserisci l'IP della macchina target: ")
5  porta_target = int(input("Inserisci la porta UDP della macchina target: "))
6  num_pacchetti = int(input("Quanti pacchetti da 1 KB vuoi inviare? "))
7
8  sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
9
10 print(f"\n[*] Invio di {num_pacchetti} pacchetti UDP da 1 KB a {ip_target}:{porta_target}")
11 print("-" * 60)
12
13 for i in range(num_pacchetti):
14     pacchetto = random.randbytes(2048)
15     sock.sendto(pacchetto, (ip_target, porta_target))
16     print(f"[+] Pacchetto {i + 1}/{num_pacchetti} inviato ({len(pacchetto)} byte)")
17 sock.close()
18 print("-" * 60)
19 print(f"[✓] Invio completato: {num_pacchetti} pacchetti inviati a {ip_target}:{porta_target}")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(kali㉿kali) - [~/Scrivania/Visual_Studio_Code]
$ /home/kali/Scrivania/Visual_Studio_Code/venv/bin/python /home/kali/Scrivania/Visual_Studio_Code/venv/udp.py
[+] Pacchetto 969/1000 inviato (2048 byte)
[+] Pacchetto 970/1000 inviato (2048 byte)
[+] Pacchetto 971/1000 inviato (2048 byte)
[+] Pacchetto 972/1000 inviato (2048 byte)
[+] Pacchetto 973/1000 inviato (2048 byte)
[+] Pacchetto 974/1000 inviato (2048 byte)
[+] Pacchetto 975/1000 inviato (2048 byte)
[+] Pacchetto 976/1000 inviato (2048 byte)
[+] Pacchetto 977/1000 inviato (2048 byte)
[+] Pacchetto 978/1000 inviato (2048 byte)
[+] Pacchetto 979/1000 inviato (2048 byte)
[+] Pacchetto 980/1000 inviato (2048 byte)
[+] Pacchetto 981/1000 inviato (2048 byte)
[+] Pacchetto 982/1000 inviato (2048 byte)
[+] Pacchetto 983/1000 inviato (2048 byte)
[+] Pacchetto 984/1000 inviato (2048 byte)
[+] Pacchetto 985/1000 inviato (2048 byte)
[+] Pacchetto 986/1000 inviato (2048 byte)
[+] Pacchetto 987/1000 inviato (2048 byte)
[+] Pacchetto 988/1000 inviato (2048 byte)
[+] Pacchetto 989/1000 inviato (2048 byte)
[+] Pacchetto 990/1000 inviato (2048 byte)
[+] Pacchetto 991/1000 inviato (2048 byte)
[+] Pacchetto 992/1000 inviato (2048 byte)
[+] Pacchetto 993/1000 inviato (2048 byte)
[+] Pacchetto 994/1000 inviato (2048 byte)
[+] Pacchetto 995/1000 inviato (2048 byte)
[+] Pacchetto 996/1000 inviato (2048 byte)
[+] Pacchetto 997/1000 inviato (2048 byte)
[+] Pacchetto 998/1000 inviato (2048 byte)
[+] Pacchetto 999/1000 inviato (2048 byte)
[+] Pacchetto 1000/1000 inviato (2048 byte)
-----
[✓] Invio completato: 1000 pacchetti inviati a 192.168.56.1:443

```

8. Conclusione

L'esercizio ha permesso di comprendere in modo pratico il funzionamento del protocollo UDP e la sua implementazione in Python tramite il modulo socket. A differenza di TCP, UDP non richiede una connessione preventiva, rendendolo ideale per test di rete ad alta velocità.

Linguaggio	Python 3 con moduli socket e random della libreria standard
Verifica	Ho catturato e confermato i pacchetti con Wireshark sul mio Windows
Rete usata	VirtualBox Host-only: Kali 192.168.56.102 verso Windows 192.168.56.1
Frammentazione	Ho rilevato con Wireshark che pacchetti >1500 byte vengono frammentati dal livello IP
Competenze	Ho acquisito competenze su socket UDP, byte casuali, cicli, input utente e debug